

Individual Contribution Reflection

Tom Tian (540180866)

COMP4347 / COMP5347 Assignment 2 - MERN Quiz Game

Subsystem: Integration, Validation, Robustness, Shared Shell, Documentation, and Tests

1. Subsystem ownership. I owned the group-of-four integration and validation role. I was not the primary owner of auth, quiz business logic, leaderboard data, or admin CRUD. My work made those parts behave as one MERN application through shared app wiring, response envelopes, error handling, route documentation, theme/shell integration, tests, and final submission evidence.

2. Major technical challenge. The hardest issue was a contract bug around Review Mode and shuffled options. The user submits the visible selectedAnswer index, but the backend must score against the original correct answer and later reconstruct the same option order in review/history. I fixed the backend test path so it starts a quiz, reads the shuffled options, submits the visible index, and asserts the saved review payload matches runtime behavior. Later commits strengthened the same contract with signed attempt tokens and persisted optionOrder.

3. Mermaid sequence diagram.

```
sequenceDiagram
    participant Player as React App
    participant API as Express App
    participant MW as Auth/Role/Rate Middleware
    participant Quiz as Quiz Controller
    participant DB as MongoDB
    Player->>API: start quiz / submit visible answers
    API->>MW: JWT, player-only, rate-limit checks
    MW->>Quiz: continue with verified user
    Quiz->>DB: sample questions or save Score answers
    DB-->>Quiz: questions/attempt data
    Quiz-->>API: score and review[] payload
    API-->>Player: shared success/error envelope
```

4. Review Mode design reflection. My design focus for Review Mode was integrity, not ownership of the feature UI. Review Mode should not alter scoring; it should faithfully explain the completed attempt. That means start, submit, saved Score.answers, and history detail must agree on question IDs, selected visible indexes, correctness, explanations, and option order.

5. Cross-system understanding. This subsystem depends on the full MERN stack: React collects input and displays feedback, Express exposes protected REST routes, middleware enforces role and validation rules, Mongoose persists the relevant state, and the shared response envelope keeps frontend error handling predictable.

6. Git commit history analysis. The table below cites meaningful commits from the GitHub Enterprise history and explains what each contributed technically.

Commit hashes: 7a61d82 eb118b5 6686eae 972fc77 160ac65 bba8b06 4a8fe22 d0d07f6 73d24c6 fd08cc8 06697e1 a6e4ad7 24ad7e7 f5d3d98

Hash	Focus	Evidence
7a61d82	Bootstrap	Established shared project layout and scripts.
eb118b5	App shell	Added shared React shell and route layout.
6686eae	D docs/tests	Added integration seed docs and tests.
972fc77	QA alignment	Aligned docs/tests with delivered API behavior.
160ac65	Risk note	Documented leaderboard/auth QA risk without crossing ownership.
bba8b06	Shuffle tests	Locked visible selectedAnswer scoring and review payload tests.
4a8fe22	Swagger	Aligned leaderboard auth API docs.
d0d07f6	Quiz errors	Documented quiz route error responses.
73d24c6	Attempt integrity	Added signed attempt tokens, replay protection, and frontend tests.
fd08cc8	Admin boundary	Enforced admin login/player route boundaries.
06697e1	Navigation guard	Guarded active quiz navigation on refresh/back/internal links.
a6e4ad7	Rate limits	Separated register limiter and exposed API root.
24ad7e7	New run flow	Unified quiz new-run actions across result/history/review.
f5d3d98	Final test lock	Locked shuffled answer mapping before final merge.

7. Final reflection. My contribution was not only a list of commits; it was a bounded subsystem responsibility with evidence. The commits above show implementation, validation, integration, and final submission hardening. In the demo I can explain both my own files and how my subsystem interacts with authentication, quiz attempts, admin data, Review Mode, and MongoDB persistence.